🧪 7.5 Hands-on Lab — Automate Model Retraining and Redeployment with Transfer Learning

Objective

By the end of this lab, learners will have:

- A **scheduled job** that monitors a data bucket for new training data
- A **training job** that fine-tunes a pretrained NVIDIA model with TAO Toolkit
- An **optimization step** with TensorRT
- A **Kubernetes + Helm deployment** that redeploys the updated model automatically

Prerequisites

- **Kubernetes cluster** with GPU nodes (Cloud: AWS EKS, Azure AKS, or GCP GKE; On-prem: DGX or GPU server)
- **NVIDIA device plugin** installed in Kubernetes
- **Helm** installed on local machine and in cluster
- **NGC account** and API key
- **Docker** with NVIDIA runtime enabled
- **TAO Toolkit** installed locally or as a Docker container
- **TensorRT** available for model optimization
- Cloud **object storage** (AWS S3, Azure Blob, or GCS) for dataset storage
- **Prometheus & Grafana** (optional) for monitoring

Step 1 — Set Up NGC and TAO Toolkit

```
# Authenticate with NGC
ngc config set
# Follow prompts to enter your NGC API key

# Pull TAO Toolkit container
docker pull nvcr.io/nvidia/tao/tao-toolkit:5.0.0-pyt
```

Explanation:

We connect to NGC and pull the TAO Toolkit container for transfer learning.

Step 2 — Create a Data Watch Script

```python
# watch_dataset.py
import boto3, time, subprocess

BUCKET_NAME = "my-training-dataset"
LAST_COUNT = 0
CHECK_INTERVAL = 600  # seconds

s3 = boto3.client('s3')

while True:
    response = s3.list_objects_v2(Bucket=BUCKET_NAME)
    count = len(response.get('Contents', []))
    if count != LAST_COUNT:
        print("New data detected. Triggering retraining...")
        subprocess.run(["kubectl", "create", "job", "--from=cronjob/train-job", "train-job-now"])
        LAST_COUNT = count
    time.sleep(CHECK_INTERVAL)
```

Explanation:

This script checks an S3 bucket for new files. If new data is found, it triggers the retraining job in Kubernetes.

Step 3 — Kubernetes CronJob for Training

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: train-job
spec:
  schedule: "0 0 * * *"  # daily at midnight
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
          - name: train-container
            image: nvcr.io/nvidia/tao/tao-toolkit:5.0.0-pyt
            command: ["bash", "-c"]
            args:
              - tao yolo_v4 train \
                -r /workspace/results \
                -k nvidia_tlt \
                -e /workspace/specs/train_spec.txt \
                --gpus 1
            resources:
              limits:
                nvidia.com/gpu: 1
```

Explanation:

This **Kubernetes CronJob** runs daily and retrains the model using TAO Toolkit. It can also be triggered manually by the watch script.

Step 4 — Optimize with TensorRT

After training completes, run optimization as a Kubernetes Job:

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: optimize-model
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: optimize
        image: nvcr.io/nvidia/tensorrt:23.05-py3
        command: ["bash", "-c"]
        args:
          - trtexec --onnx=/workspace/model.onnx \
                    --saveEngine=/workspace/model.trt \
                    --fp16
        resources:
          limits:
            nvidia.com/gpu: 1
```

Explanation:

We run TensorRT to produce an optimized `.trt` engine ready for inference.

Step 5 — Push Model to NGC or Private Registry

```
# Tag and push container with new model
docker build -t registry.ngc.nvidia.com/my-team/my-model:latest .
docker push registry.ngc.nvidia.com/my-team/my-model:latest
```

Explanation:

We build a new inference container (with Triton or custom app) containing the optimized model and push it to a registry.

Step 6 — Helm Deployment

`values.yaml` (snippet):

```yaml
image:
  repository: registry.ngc.nvidia.com/my-team/my-model
  tag: latest
resources:
  limits:
    nvidia.com/gpu: 1
```

Deploy or upgrade:

```
helm upgrade --install ai-inference ./helm-chart
```

Explanation:

Helm deploys the updated model into the Kubernetes cluster. Using `helm upgrade` ensures zero-downtime rollout.

Step 7 — Automate Full Pipeline

- **Option A:** Use Argo Workflows to chain:
  Data Watch → Training Job → Optimization Job → Helm Upgrade
- **Option B:** Use GitOps with ArgoCD to trigger Helm redeploy when new image is pushed
- **Option C:** Combine with CI/CD tools like GitHub Actions or GitLab CI

Step 8 — Verify Deployment

- Send test inference requests to Triton server endpoint:

```
curl -X POST http://<NODE_IP>:8000/v2/models/my-model/infer -d @input.json
```

- Check Kubernetes logs:

```
kubectl logs -f deployment/ai-inference
```

Step 9 — Monitor and Rollback

- Monitor inference latency, throughput, and GPU usage via Prometheus + Grafana
- Rollback with:

```
helm rollback ai-inference <REVISION_NUMBER>
```

Step 10 — Optional: Edge Rollout

If using **Fleet Command**, push the new container to edge devices after Kubernetes validation.
